이벤트 기반 인디언 포커 멀티 에이전트 시스템

선택 과제 2번

주제: 여러 에이전트가 동시에 대화하며 보드게임을 할 수 있는 시스템 개발

G084 신동범

여러 LLM 에이전트와 인간이 같은 테이블에서 대화하고 베팅하는 인디언 포커 시스템을 구현하였다. 참가자별 판단을 독립적으로 실행하고 공용 사건 큐로 연결하였으며, 개인 메모와 공개 과거 기록 검색을 추가하였다. 실험에서는 정보에 따른 행동 변화가 확인됐지만 검색과 메모가 일관된 칩 손익 개선으로 이어지지는 않았다.

1 프로젝트 목표와 전체 구조

인디언 포커는 자신의 카드를 볼 수 없고 상대의 카드만 관찰하는 게임이다. 상대의 주장과 과거 행동을 참고할 여지가 있어, 대화와 기억 정보가 판단에 어떤 영향을 주는지 관찰하는 환경으로 선택하였다. 구현은 고정 차례의 대화보다 자유로운 발언과 인간 참여에 중점을 두었다.

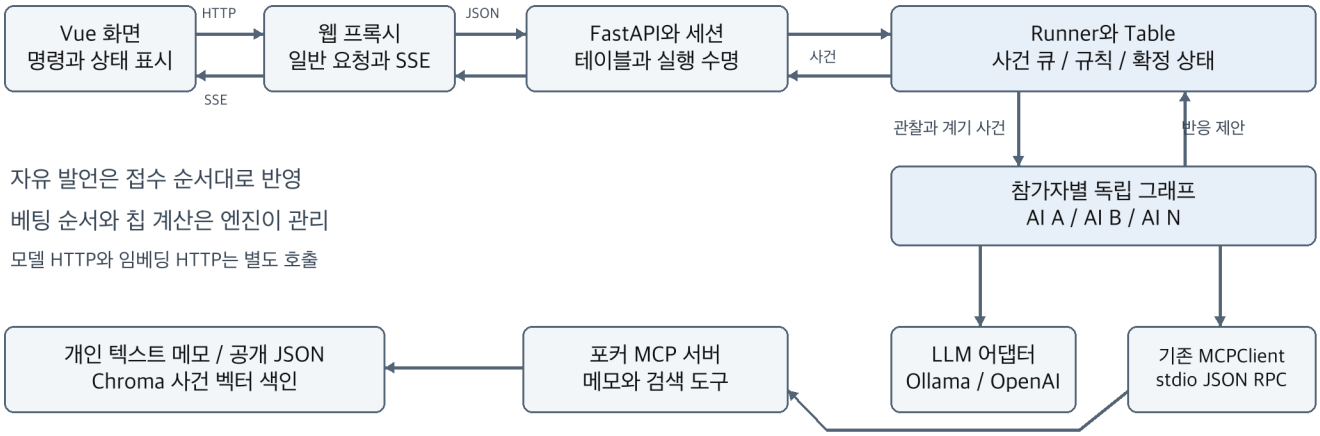


그림 1 전체 시스템의 책임과 통신 경계

Vue 화면은 입력과 상태 표시를 담당하고, 웹 프록시는 화면 제공과 API 중계를 담당한다. FastAPI와 세션 계층이 실행 수명을 관리하며 Runner가 참가자 작업을 조정한다. Table과 규칙 객체가 카드, 칩, 베팅 및 확정 사건을 소유한다. 참가자 그래프는 관찰에 근거한 반응만 제안하고 모델과 기억 도구는 별도 어댑터로 호출한다.

기존 코드의 재사용

설계의 출발점은 주어진 인프라를 최대한 유지하는 것이었다. 기존 mcp_client와 ollama_client를 재사용하고 게임별 코드를 agents 모듈에 모았다. OpenAI 지원은 공통 LLM 인터페이스 아래에 추가하여 그래프의 판단 흐름을 유지하였다. 서버와 웹 프록시의 역할은 보존하되 게임 API를 연결하였고, 화면은 컴포넌트 관리를 위해 Vue로 구성하였다. 현재 실행에서 사용하지 않는 옛 의존성 파일까지 재사용한 것으로 보지는 않았다.

2 참가자 그래프와 메시지 큐

모든 AI는 같은 그래프 구성을 사용하되 참가자별 판단 객체를 주입받는다. 한 번의 그래프 실행은 게임 전체가 아니라 현재 사건에 대한 반응 하나를 만든다. 그래프 상태는 해당 호출의 작업 상태이며 다음 호출은 최신 관찰과 외부 메모로 다시 구성한다.

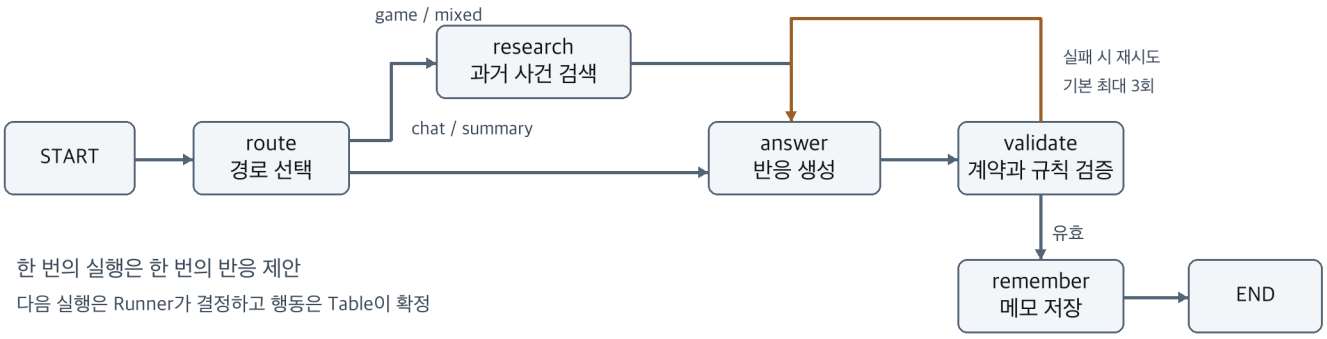


그림 2 참가자 한 명의 판단 흐름과 검증 재시도

route는 게임 반응, 일반 질문, 혼합 응답, 공개 후 요약을 구분한다. 게임과 혼합 경로는 research에서 관련 경험을 찾은 뒤 answer로 진행한다. 일반 질문과 요약은 검색을 생략한다. validate는 형식뿐 아니라 현재 사건, 발언 가능 여부, 허용 베팅을 확인한다. 오류가 있으면 피드백을 붙여 재시도하고 유효한 메모를 저장한 뒤 종료한다.

인간의 복합 발언은 별도의 SpeechInterpreter가 한 번 해석하여 의도, 대상, 원문 조각으로 나눈다. 참가자 그래프의 route는 이 검증된 해석을 활용하는 코드 분기이다. 예를 들어 일상 질문은 지목된 참가자에게 보내고 게임 협상은 관련 참가자들이 받을 수 있게 한다. 카드 질문은 게임상의 주장으로 다루며 모델 입력에서는 인간과 AI의 구분을 제거해 특정 참가자를 도우미의 사용자로 특별 취급하지 않도록 하였다.

독립 실행과 공용 상태의 확정

A와 B의 응답을 한 묶음으로 실행한 뒤 모두 끝날 때까지 기다리면 대화가 일괄 답변처럼 움직일 수 있다. 이를 피하기 위해 Runner는 참가자별 비동기 작업을 독립적으로 실행한다. 인간 입력과 AI 작업 완료는 프로세스 내부 asyncio.Queue로 접수한다. 같은 참가자의 판단은 하나만 진행하고 다른 참가자들의 작업은 함께 진행될 수 있게 하였다.

판단 중 새 사건이 오면 최신 계기를 보류하고 지목된 질문은 수신함에 보존한다. 다음 호출에서 관찰을 다시 구성하며 오래된 제안은 Table이 거절하거나 재판단하게 한다. 이 큐는 외부 메시지 브로커가 아니며 여러 작업이 존재한다고 로컬 모델의 실제 추론이 모두 병렬로 수행되는 것은 아니다.

발언은 접수 순서대로 반영하지만 베팅은 좌석 순서를 따른다. 칩 계산과 합법적인 레이즈 선택지는 엔진이 제공한다. AI의 연속 게임 발언은 보류하고 인간의 연속 발언은 허용하였다. 폴드 후에도 남은 기회로 발언할 수 있으며 라운드 종료 후에는 발언을 금지한다. 이러한 구조의 자연스러움과 사용자 경험이 순차 대화보다 나은지는 별도 비교가 필요하다.

3 기억 구조와 통신 설계

개인 메모와 공개 경험 검색

개인 메모는 이번 라운드, 이번 게임, 상대별 장기 대응으로 나눈 텍스트 파일이다. LLM 호출은 이전 상태를 보존하지 않으므로 세 메모를 매 호출에 첨부한다. 기본 상한은 각각 1,000자, 2,000자, 3,000자이며 현재 대화도 최근 메시지 수를 제한한다. Table이 도구 호출에 출처 참가자를 주입하여 다른 참가자의 메모를 지정하지 못하게 한다.

공개 경험은 완료 라운드의 원본 JSON과 Chroma 색인으로 분리하고 기본 최근 20라운드를 보존한다. 발언 중 매번 동기 임베딩하는 부담을 피하려고 라운드 종료 후 백그라운드에서 색인한다. embeddinggemma:latest를 사용하며 사건별 청킹을 적용하였다. 기본 예산은 헤더를 포함한 UTF-8 1,536바이트이고, 실제 2,048토큰 문맥을 넘는 청크는 추가 분할한다. 정확한 토크나이저 기반 분할이나 자동 잘림은 사용하지 않는다.

주장 신뢰성은 Speech, 베팅 성향은 BetPlaced를 상대 ID와 함께 필터링하여 각각 기본 5개 사건을 회수한다. 사건 ID로 청크 중복을 제거하고 같은 과거 라운드의 공개 결과를 연결한다. 폴드로 공개되지 않은 카드는 미확인으로 둔다. 인간의 혼합 입력은 게임 관련 원문 조각을 검색과 색인에 사용한다. 검색기는 사례를 제공하며 신뢰 점수나 최적 전략을 직접 계산하지 않는다.

API와 도구 통신

브라우저 명령은 HTTP JSON으로 제출하고 확정 사건은 SSE로 구독한다. 인간 입력은 기존 요청으로 충분하고 서버에서 생기는 발언과 베팅은 단방향으로 전달하면 되므로 이 방식을 택하였다. SSE는 모델 토큰이 아닌 게임 사건을 전달하며 커서, keep-alive, 재연결을 지원한다. 프록시는 일반 응답과 긴 SSE 연결을 분리하여 종료 시 관련 연결을 정리한다.

MCPClient는 도구 서버를 하위 프로세스로 실행하고 stdio의 JSON-RPC 2.0으로 통신한다. 메모 읽기와 쓰기, 공개 경험 검색, 색인 상태 확인을 제공하며 완료 경험 저장은 Table만 수행한다. 모델 호출은 Ollama의 chat API 또는 OpenAI Responses API, 임베딩은 Ollama embed API를 사용한다. 실제 백엔드와 모델은 config 및 llm_controller에서 선택한다.

계약	반드시 담아야 할 정보
관찰 Observation	관찰자와 기준 사건, 공개 카드, 칩과 베팅 상태, 허용 행동, 발언 여유와 최근 대화
명령과 결정	요청 또는 기준 사건 ID, 발언과 베팅 제안, 대상과 응답 연결, 개인 메모 갱신
해석과 적용 결과	의도와 대상 및 원문 조각, 수락 또는 거절 및 보류 사유, 재판단 여부

외부 입력과 모델 결과는 Pydantic으로 검증하고 현재 상태와 다시 대조한다. 단순 카드와 내부 객체에는 dataclass를 사용하였다. 생성자와 함수 인자로 의존성을 주입하여 모델과 도구를 교체하며, LangSmith는 라운드와 참가자별 그래프를 중심으로 추적한다. 실행 로그에는 입력 해석, 선택 경로, 판단 결과와 최종 적용을 남긴다.

4 실험 방법과 최종 결과

실험의 목적은 과거 발언과 베팅, 개인 메모를 제공했을 때 판단과 근거가 달라지는지, 실제 칩 손익도 개선되는지, 이를 위해 어느 정도의 모델 능력이 필요한지 확인하는 것이었다. 웹 UI를 거치지 않고 기존 Table, Runner, 그래프와 MCP 경로를 사용하였다.

1단계는 관련 이력과 무관 이력에 따른 단일 결정의 변화, 2단계는 정보 없음과 검색만, 메모만, 둘 다의 차이를 살폈다. 3단계는 라운드를 끝까지 실행해 승패와 손익을 측정하였다. 4단계는 정직 이력과 기만 이력을 직접 제공해 검색 품질과 모델의 해석 능력을 분리하였다. 일반 대화는 모든 조건에 있어 대화 유무의 효과는 따로 측정하지 않았다.

최종 고정 배패 평가

최종 3단계는 gpt-6.1-sol 참가자 A와 고정 규칙 상대 B 또는 C의 1대1 라운드이다. B는 거짓 범위를 말하는 공격형, C는 사실에 맞는 범위를 말하는 수비형이며 둘 다 LLM과 기억 도구를 사용하지 않는다. 배패 10개를 두 상대와 네 정보 조건에 적용한 총 80라운드이다. 시행마다 칩 30개로 시작하고 Chroma와 메모를 격리했으며 상대별 합성 과거 20라운드는 동일하게 유지했다.

상대	정보 없음	검색만	메모만	검색과 메모
B	60% / +1.6	50% / +1.4	60% / +4.0	50% / +1.2
C	60% / +0.8	60% / +0.5	60% / +0.5	60% / +0.5

표 1 최종 상대별 승률과 평균 칩 손익. 각 셀은 10라운드이다. 승리는 라운드 판돈 획득이며 손익은 종료 칩에서 초기 30칩을 뺀 값으로 참가비를 포함한다.

80라운드 모두 유효하게 종료했지만 정보 제공의 일관된 우위는 없었다. B의 메모만 조건은 기준보다 총 24칩 높았으나 배패 08의 +20칩과 배패 09의 +4칩에 집중됐다. 나머지 여덟 배패는 같았다. C에서는 승률이 같았고 배패 09 한 건의 베팅 크기가 손익 차이를 만들었다. 평균의 상승만으로 전반적인 전략 향상을 단정하기 어렵다.

최종 검색에서는 주장 200건과 베팅 310건이 의도한 상대와 종류로 회수되었다. 그러나 검색 구성이 정상이라는 사실이 모델의 적절한 활용을 보장하지는 않는다. 4단계에서는 정직 이력과 기만 이력 사이에 10개 배패 중 8개에서 행동 종류가 달라졌다. 적절히 제시한 정보가 행동에 영향을 줄 수 있음은 확인했지만 단일 결정 실험이므로 손익 향상을 보여주지는 않는다.

다양한 단계에 대해 Qwen3:8b, gpt-6-luna, gpt-6.1-sol등의 모델로 실험한 결과, 모델의 성능이 약할수록 한두 행동에 매몰되어 반복하는 현상 및 필요한 정보를 활용하지 못하는 현상을 관측하였다. 이를 해결하기 위해 높은 성능의 sol 모델을 활용할 수 밖에 없었다. 그에 따라 실험 비용이 유의미한 크기로 발생함을 인지하였다.

모델 교체 과정에서는 레이즈 편향과 카드 소유자 혼동이 관찰되었다. Sol에서도 수익의 일관된 개선은 없어 보편적인 최소 모델 성능은 확정하지 못했다. 최종 실행은 약 32분 52초, LLM 368회, 기록 토큰 1,136,305개였으며 모델 왕복 평균은 약 5.05초였다. 고정 배패별 한 번의 평가이고 유의성 검정과 장기 게임 평가는 하지 않았으므로 결과의 일반화에는 한계가 있다.

5 구현 결과와 개발 경험

구현한 기능과 관측 범위

시스템은 여러 AI의 독립 반응과 선택적 인간 참가를 하나의 테이블에 연결한다. 화면은 원형 테이블의 캐릭터, 카드, 말풍선, 베팅 입력과 사건 목록을 컴포넌트로 나누었다. 현재 게임에서는 과거 사건을 선택해 해당 시점의 화면을 볼 수 있다. 이 기능은 저장된 화면 상태를 사용하므로 모델을 다시 호출하지 않는다. 서버 재시작을 넘어 임의의 게임을 완전히 재생하거나 실행을 재개하는 기능은 아직 포함하지 않았다.

Pydantic과 엔진 검증으로 허용되지 않은 행동을 걸러 내고, 기준 사건 ID와 요청 ID로 오래된 판단과 중복 적용을 처리하였다. 모델이나 통신 오류로 참가자가 실패하면 상태를 기록하고 비용이 없을 때는 체크, 추가 비용이 필요하면 폴드한다. 실행 로그와 LangSmith에서는 참가자별 경로와 결과를 구분하여 관찰할 수 있다. 다만 동작 가능한 시스템을 구현한 것과 자유 대화 방식의 우수성을 입증한 것은 구분해야 한다.

개발 과정에서 느낀 점

가장 어려웠던 점은 LLM이 의도한 방식으로 행동하도록 유도하는 일이었다. 허용 행동을 스키마로 제시하고 목표를 설명해도 특정 행동을 반복하거나 지시문의 의미를 다르게 받아들였다. 지금 행동을 고르라는 요구를 레이즈하라는 요구로 해석하지 않도록 선택의 의미까지 구체적으로 설명해야 했다.

사람이 직관적으로 필요하다고 판단한 정보를 넣는 것과 모델이 그것을 잘 사용하는 것은 별개였다. 과거 거짓말과 공개 결과를 주면 신뢰성을 판단할 것이라 예상했지만 카드 소유자를 혼동하거나 근거를 기록하고도 행동이 달라지지 않았다. 반대로 행동이 달라졌더라도 올바른 근거 때문인지, 이익이 되는지는 쉽게 알 수 없었다. 이 점이 기능 구현 이상의 개발 부담이 되었다.

개선 방향

우선 위험 선호와 발언 성향을 분리하고, 게임에 맞춘 적극적인 프롬프팅과 전략 예시를 제공하여 행동의 차이를 분명하게 만들 수 있다. 이후 새로운 배패에서 검증해 특정 사례에만 맞춘 지시문이 되지 않도록 해야 한다.

기억 정보는 유사 사례 외에도 공개 결과로 확인할 수 있는 주장 일치 비율과 베팅 분포를 요약할 수 있다. 미공개 카드를 임의로 판정하지 않고 자료 회수와 모델 해석을 나누어 평가해야 한다. 리랭킹은 이러한 정리의 효과를 확인한 뒤 도입 여부를 판단하는 것이 적절하다.

나아가 에이전트별 페르소나와 전략을 설정할 수 있다. 지금은 모든 에이전트가 동일한 프롬프트를 공유하고 있으나, 다양한 게임 플레이 성향 및 가치관, 지식 등을 설정하여 게임으로서의 완성도를 높일 수 있다.

장기적으로는 참가자 관리, 사건 전달, 인간 입력과 모델 및 도구 어댑터를 공통 기반으로 두고 관찰 생성, 명령 검증, 상태 전이와 승패 규칙을 교체 가능하게 만들 수 있다. 현재 구현은 포커에 특화되어 있으므로 범용 프레임워크로의 변환은 후속 과제이다. 추가 평가는 새로운 배패와 실제 이력, 반복 시행, 순차 대화와의 사용자 경험 비교를 포함해야 한다.

부록 A 구현 위치와 인터페이스

기준일은 2026년 10월 22일이다. 아래 위치는 해당 시점의 코드이며 줄 번호는 이후 수정에 따라 달라질 수 있다. 역사적 실험은 실행 당시 manifest의 설정과 소스 해시를 기준으로 구분한다. 본문은 1쪽부터 5쪽까지이며 상세 구현과 통계는 이 부록부터 제시한다.

A1 기술별 파일과 심벌

기술과 역할	파일	심벌과 시작 줄
멀티 에이전트 스케줄링	agents/runner.py	RoundRunner._dispatch :63 RoundRunner._launch :91 RoundRunner.run_round :225
상태 확정, 행동 적용	agents/table.py	Table.submit :87 Table.observe :281 Table.apply_decision :332
라운드 생명주기	agents/round.py	Round :9
베팅 규칙	agents/betting.py	Betting :5
발언 규칙	agents/conversation.py	Conversation :6
카드 판정	agents/cards.py	Card :9 find_winner :39
참가자 그래프	agents/graph.py	build_graph :126 validate_node :66
판단, 검색 조립	agents/poker_llm.py	PokerLLM.route :42 PokerLLM.research :57 PokerLLM.decide :170
메모 자동 첨부	agents/player_llm.py	PlayerLLM._system :20 PlayerLLM.complete :38
복합 인간 입력	agents/speech_interpreter.py	SpeechInterpreter.interpret :16
MCP 클라이언트 재사용	agent/mcp_client.py	MCPClient.start :53 MCPClient.call_tool :71
메모, 공개 경험, Chroma	agents/mcp_server.py	GameTools.read_memories :81 GameTools.index_records :236 GameTools.search_experiences :277
청킹, 게임 발언 추출	agents/chunker.py	game_speech_text :9 EventChunker.split_text :88 Chunk.metadata :54
도구 권한 경계	agents/table.py	Table.call_player_tool :372 Table.publish_memories :384
모델 교체	agent/llm_controller.py	LLMClient :14 create_llm :45
Ollama 어댑터	agent/ollama_client.py	OllamaLLM :9
OpenAI 어댑터	agent/openai_client.py	structured_format :12 OpenAILLM :44
계약과 입력 검증	agents/models.py	PlayerCommand :63 AgentDecision :92 Observation :120
설정 분리	agents/config.py	GameConfig :9
서비스, 사건 저널	agents/game_service.py	EventJournal.stream :56 GameSession :77 GameService :219

기술과 역할	파일	심벌과 시작 줄
FastAPI, DI	server.py	create_app :60 create_app.session :108 create_app.events :150
웹 프록시	web_demo_server.py	proxy_health :43 proxy_events :68 proxy_games :112
LangSmith 초기화	agents/tracing.py	configure_tracing :10
실행 진단 로그	agents/runtime_log.py	record_runtime :7
현재 게임의 과거 화면	agents/replay_history.py	ReplayHistory :7
실험, 노드 시간	experiments/common.py	RunRecorder :70 NodeTimingHandler :159
실험 실행 경계	experiments/execution.py	run_trial :68
고정 상대	experiments/rule_opponents.py	BRulePolicy :61 CRulePolicy :84
Vue 세션, 입력, 구독	useGameSession.js	useGameSession :11
브라우저 SSE	eventStream.js	subscribeGameEvents :5
HTTP 클라이언트	gameApi.js	createGameApi :15
과거 화면 선택	useReplay.js	useReplay :6
캐릭터 컴포넌트	SlimeAvatar.vue, AvatarEyes.vue	SFC 전체
프롬프트 분리	agents/prompts/	각 템플릿

의존성 주입은 build_graph(backend), RoundRunner(table, graphs, on_event, interpreter), PokerLLM(table, mcp, player_id, llm), PlayerLLM의 client, create_app의 config와 mcp_factory 인자로 구현한다. 별도 대형 DI 컨테이너는 사용하지 않았다. FastAPI Depends는 요청별 인증과 세션 조회에 사용한다.

재사용 범위는 기존 MCP 프로세스와 클라이언트, Ollama 호출 기반이다. OpenAI 어댑터와 게임 업무 계층은 확장 또는 신규 구현이며 서버와 프록시는 기존 역할을 유지하면서 게임 경로를 연결하였다. _old의 deps, retry, session_store는 현재 활성 코드에서 참조하지 않는다.

A2 API와 MCP 도구

일반 API는 HTTP JSON이며 테이블 생성 시 발급한 Bearer 접근 토큰을 사용한다. 게임 명령과 사건 스트림의 수명을 구분하였다. 기본 백엔드 포트는 8100, 웹 프록시는 8600이다.

메서드와 경로	주요 역할
GET /health	백엔드 런타임과 설정 확인. 프록시는 자체 화면 파일과 백엔드 health를 함께 확인
POST /games	AI ID와 선택적 인간 ID로 생성하고 접근 토큰과 상태 반환
GET /games/{id}	현재 관찰 상태 조회
GET /games/{id}/history	현재 게임의 라운드 목록 또는 사건 직후 화면 조회
POST /games/{id}/rounds	라운드 백그라운드 실행 시작. 202 응답
POST /games/{id}/commands	발언, 발언 없음, 베팅 제출
POST /games/{id}/commands/release	기존 보류 발언 재확인 호환 경로
GET /games/{id}/events	커서 기반 SSE 구독
DELETE /games/{id}	게임과 관련 작업 종료

SSE는 id에 전송 sequence, event에 사건 종류, data에 JSON을 담는다. Table의 event_id와 전송 커서는 구분된다. 기본 2,000개 사건과 15초 keep-alive를 사용하며 오래된 커서에는 재동기화를 요구한다. 브라우저는 인증 헤더를 전달하기 위해 EventSource 대신 fetch reader를 사용한다. 같은 출처의 프록시 경로를 제공하며 별도의 CORS 미들웨어를 추가한 구성은 아니다.

MCP 도구	자료와 사용 범위
read_memo	참가자의 round, game, long 중 한 범위 읽기
read_memories	세 범위를 함께 읽어 모델 입력 구성
write_memo	범위별 텍스트 교체 또는 추가. 길이 제한 적용
search_experiences	질의와 상대 ID 및 claims/betting 종류로 검색
memory_status	보관 및 색인 상태 확인
store_experience	완료 라운드 저장과 색인 요청. Table 전용

MCP는 subprocess stdio의 JSON-RPC 2.0을 사용한다. 도구의 game_id, agent_id, round_id는 Table이 주입한다. 실제 사용 경로는 코드가 정한 research와 remember 호출이며 모델이 임의의 도구를 반복 선택하는 실행 루프는 아니다. 일반 검색 도구의 limit 기본값 3과 포커 경로의 종류별 기본값 5는 구분한다.

A3 데이터 계약과 운영 설정

계약	필요한 정보와 검증 경계
PlayerCommand	요청 ID, 라운드와 기준 사건, speak/silent/bet, 발언 또는 Bet. 중복과 잘못된 조합 검사
Observation	관찰자, 기준 사건, 참가자, 볼 수 있는 카드, 판돈과 기여액, 베팅 차례와 선택지, 발언 수와 대화, 공개 결과
DecisionDraft	모델이 선택한 반응, 베팅과 대기, 응답 채널, 메모 초안. 출처 식별자는 실행 코드가 부착
AgentDecision	관찰에 대한 반응 제안, 대상과 응답 연결, MemoryUpdate. 실제 적용은 Table이 수행
SpeechInterpretation	game/chat/private_request 의도, 대상과 원문 조각. 실제 입력과 대상 존재 여부 검사
Bet	check/call/fold/raise와 이름형 raise_size. 금액 계산은 엔진이 담당
MemoryUpdate	round/game/long 교체 텍스트. 빈 문자열은 기존 메모 유지
CommandResult	수락, 거절, 보류 상태와 사유, 적용 사건, 재판단 여부

계약은 strict=True와 extra=forbid를 적용한다. JSON Schema 제공과 파싱 이후 그래프와 Table에서 상태 규칙을 다시 검사한다. 그래프 입력과 작업 상태는 TypedDict이며 Player, Card, GameConfig 등은 dataclass이다.

설정	현재 기본값
게임	초기 칩 30, 참가비 2, 목표 칩 고정 60, 대출과 사이드팟 없음
발언과 문맥	발언 최대 5, 최소 1, 최근 메시지 최대 40
기억	공개 경험 최근 20라운드, 메모 1,000/2,000/3,000자
모델	OpenAI gpt-6.1-sol. OpenAI 해석 모델 미지정 시 같은 모델 사용
Ollama 선택 시	판단 qwen3:8b, 해석 qwen3.5:2b
임베딩과 검색	embeddinggemma:latest, 청크 1,536바이트, 주장과 베팅 각각 5개 사건
재시도와 오류	그래프 검증 기본 3회. 실패 참가자는 무료 체크 또는 비용이 필요하면 폴드

숫자는 높을수록 강하고 A는 K를 이기지만 2는 A를 이긴다. 같은 숫자는 S, D, H, C 순으로 강하다. 순환 관계에서 단일 승자가 없으면 무승부로 판돈을 이월한다. 폴드한 카드는 공개하지 않고 다음 좌석 순서는 한 칸 회전한다. 다음 참가비를 낼 수 없는 참가자는 탈락한다.

개인 메모는 텍스트이고 공개 경험은 원본 JSON과 Chroma 색인이다. 청크 메타데이터에는 경험, 출처, 라운드, 사건, 발언자, 종류, 사건 키, 청크 순서와 개수, 색인 스키마 버전을 담는다. 같은 사건의 청크는 회수 시 묶지만 다른 라운드의 같은 문장은 별도 사건으로 남는다.

LangSmith는 루트 .env를 읽어 설정하고 라운드 상위 trace와 참가자별 그래프 trace를 연결한다. 메타데이터에는 게임, 라운드, 참가자, 기준과 계기 사건을 담는다. Chroma 내부를 별도 추적하지 않는다. runtime.jsonl에는 해석과 경로, 결과, 실패 및 적용을 남긴다. 최근 인간 동등 취급과 추적 보완은 최종 성과 실험 이후의 변경이므로 그 효과를 해당 실험 수치로 입증하지는 않는다.

부록 B 실험 설계와 상세 결과

B1 실험 단계와 데이터 격리

단계	비교 조건	평가 단위
1	정보 없음 / 관련 이력 / 무관 이력	같은 상황의 한 번의 판단과 메모 근거
2	control / search_only / memo_only / both	같은 상황의 한 번의 결정
3	A 대 B 또는 C, 위의 네 조건	독립 라운드의 승패, 칩 손익과 시간
4	정보 없음 / 정직 이력 / 기만 이력 직접 제공	검색을 제외한 한 번의 신뢰성 판단

현재 라운드 대화와 판세는 모든 조건에 포함된다. 실험 1, 2, 4는 단일 결정 실험이므로 승률을 계산하지 않는다. 실험 3의 승리는 A가 판돈을 획득한 라운드이고 손익은 종료 칩에서 시작 칩 30을 뺀 값이다. 장기 게임 우승이나 탈락률과 구분한다.

최종 실험은 같은 배패 10개를 B와 C 및 네 조건에 적용한 80라운드이다. 조건마다 별도 data/trial_id를 사용해 Chroma와 개인 메모를 초기화한다. 상대별 과거 20라운드는 seed 30042로 엔진을 통해 생성한 합성 기록이며 모든 비교 조건에서 고정된다. 평가 라운드가 다음 시행의 검색 자료로 누적되지 않는다. 메모 사용 조건은 같은 사전 요약에서 시작해 라운드 중 갱신한다.

입력, 과거 뱅크, 공개 뱅크 해시로 20개 배패와 상대 그룹의 조건 일치를 확인하였다. 최종 80개 시행은 모두 유효하며 정산과 칩 변화도 대조했다. 메모와 검색의 자료 분량은 완전히 동일하지 않으므로 정보원의 존재와 입력 표현을 함께 비교하는 절제 실험이다.

B2 최종 상대 정책

B와 C는 자신의 카드가 아니라 관찰한 A의 카드와 콜 비용 및 확정된 자신의 레이즈를 사용한다. 난수와 LLM 및 기억 도구를 사용하지 않고 라운드당 자신의 레이즈는 최대 한 번이다. B는 카드 범위를 반대로 주장하고 C는 사실대로 주장한다.

상대	관찰한 A 카드	추가 비용 없음	콜 비용 있음
B	2부터 7	4칩 우선, 불가 시 2 또는 1. 올렸으면 체크	아직 올리지 않았으면 레이즈 시도, 이후 콜
B	8부터 10	2칩 우선, 불가 시 1. 올렸으면 체크	4 이하 콜, 초과 폴드
B	J부터 A	체크	1 이하 콜, 초과 폴드
C	2부터 7	1칩 레이즈, 이미 올렸으면 체크	콜
C	8부터 10	체크	2 이하 콜, 초과 폴드
C	J부터 A	체크	폴드

B3 최종 상대별 통계와 고정 배패

상대	조건	승리	승률	총 손익	평균	중앙값
B	control	6/10	60%	16	1.6	2.0
B	search_only	5/10	50%	14	1.4	0.0
B	memo_only	6/10	60%	40	4	2.0
B	both	5/10	50%	12	1.2	0.0
C	control	6/10	60%	8	0.8	2.0
C	search_only	6/10	60%	5	0.5	2.0
C	memo_only	6/10	60%	5	0.5	2.0
C	both	6/10	60%	5	0.5	2.0

기준 실행은 20261022T001613-70345d11이며 2026년 10월 22일 09시 16분부터 09시 49분까지 KST로 실행되었다. 실제 응답 모델은 gpt-6.1-sol, reasoning 설정은 low다. 형식 재시도, 참가자 실패, 자동 대체 없이 80라운드가 종료되었다.

배패	A의 숨은 카드	상대 카드
01	10H	9C
02	AH	2S
03	2C	AH
04	KS	KD
05	9S	10C
06	QD	JH
07	JH	QS
08	4C	3D
09	5S	5H
10	6D	7C

A의 숨은 카드는 평가 정답이며 공개 전 모델 관찰에 포함하지 않는다. 표의 상대 카드는 A가 관찰한다. S는 스페이드, D는 다이아몬드, H는 하트, C는 클로버이며 A는 에이스다. 같은 10개 배패를 두 상대와 모든 정보 조건에서 사용하였다.

B4 배패별 칩 손익

배패와 상대	control	search_only	memo_only	both
s3-B-01	4	4	4	-2
s3-B-02	2	-2	2	2
s3-B-03	-2	-2	-2	-2
s3-B-04	2	2	2	2
s3-B-05	-4	-2	-4	-2
s3-B-06	2	2	2	2
s3-B-07	-2	-2	-2	-2
s3-B-08	10	6	30	10
s3-B-09	10	10	14	10
s3-B-10	-6	-2	-6	-6
s3-C-01	2	2	2	2
s3-C-02	2	2	2	2
s3-C-03	-2	-2	-2	-2
s3-C-04	2	2	2	2
s3-C-05	-2	-2	-2	-2
s3-C-06	2	2	2	2
s3-C-07	-2	-2	-2	-2
s3-C-08	3	3	3	3
s3-C-09	6	3	3	3
s3-C-10	-3	-3	-3	-3

B의 memo_only 우위 총 24칩은 B-08의 +20과 B-09의 +4에 집중된다. B-08을 제외하면 나머지 아홉 배패의 차이는 총 +4칩이고 손익 중앙값은 control과 같은 +2다. C는 세 정보 조건 모두 C-09에서만 3칩 적게 벌었다.

B의 search_only는 B-02에서 -4, B-05에서 +2, B-08에서 -4, B-10에서 +4로 control보다 총 2칩 적다. both는 B-01의 -6과 B-05의 +2로 총 4칩 적다. 행동과 승패, 실현 손익이 같은 방향으로 움직이는 것은 아니다.

B5 검색 감사와 원인 분리 실험

검색 종류	실제 호출	반환 사건	확인한 사실
claims	40	Speech 200건	ready, 올바른 상대와 사건 종류, 같은 사건 청크의 중복 자리 없음
betting	62	BetPlaced 310건	ready, 올바른 상대와 사건 종류, 같은 사건 청크의 중복 자리 없음

주장 200건의 문구는 현재 검색 문장에 포함된 주장과 같았다. 이는 과거 카드 주장 질의에 베팅만 회수되는 문제를 줄였다는 근거다. 문구 일치는 해석의 정확성이나 전체 이력의 대표성을 보장하지 않는다. 이 실행에는 인간 발언 해석 사건이 없어 인간의 혼합 질의 분리 효과는 평가하지 않았다.

실험 1은 Qwen3:8b로 30상황과 세 조건의 90개 결정을 평가하였다. 기준 대비 관련 이력은 25/30, 무관 이력도 25/30에서 베팅 객체가 달랐다. 레이즈 크기 변화도 포함하므로 변화만으로 적절한 활용을 입증하기 어렵다. 실험 2의 120개 결정에서는 control 대비 검색만 22/30, 메모만 15/30, 둘 다 17/30이 달랐다. 이 단계는 라운드 승률 평가가 아니다.

초기의 레이즈 편향은 표준 지시문을 보존한 채 체크 지시문을 덮어써 진단했다. force_bet이 false일 때와 true일 때 모두 허용된 체크와 콜이 처리되어 엔진이 레이즈만 허용하는 것은 아님을 확인했다. force_bet은 지금 합법적인 행동을 확정하라는 뜻이며 레이즈나 추가 지출을 요구하지 않는다.

Luna에서는 과거 주장에 관한 A 카드와 C 카드를 혼동하여 올바른 초기 메모를 잘못 갱신한 사례가 있었다. 해당 사례의 손익이 같았으므로 전체 성과 저하의 직접 원인으로 확대하지 않는다. Sol 초기 연결에서는 지원하지 않는 reasoning 설정으로 4회 모두 오류가 난 실행도 있었다. 이 약 8.5초의 실패 종료를 정상 모델 응답 속도로 비교하지 않는다.

실험 4의 직접 제공 이력 비교

조건	fold	call	raise
정보 없음	4	4	2
정직 이력	6	3	1
기만 이력	1	4	5

서로 다른 배패 10개와 세 조건의 30개 결정이며 실제 Sol 응답 30회, 206.036초, 기록 95,345토큰이었다. 정직과 기만 이력 사이에 행동 종류가 8/10에서 달랐다. 정보가 있는 20개 결과의 장기 메모는 카드 소유자와 주장 진위를 올바르게 기록했고, 기존 수동 검토에서 17개는 현재 판단과 명시적으로 연결했다. 나머지의 연결 부재를 바로 미사용으로 판정하지는 않았다. 이 실험은 검색기를 제외하고 사전 메모 없이 과거 발언과 공개 결과를 직접 주었으며 단일 결정만 평가했다.

B6 버전별 성과와 시간 및 토큰

각 셀은 두 상대를 같은 비중으로 합친 20라운드의 승률과 평균 칩 손익이다. 확률 상대와 결정적 상대, 검색 및 프롬프트 버전을 혼합한 단일 표본으로 해석하지 않는다.

실행과 정책	control	검색만	메모만	둘 다
Sol 초기 확률 정책	55% / +0.20	55% / +0.20	50% / -0.40	50% / -0.20
Luna 초기 확률 정책	50% / -0.15	미실행	미실행	50% / -0.40
Sol 결정적 상대	55% / +1.05	50% / +0.55	60% / +0.85	55% / +1.75
Sol 최종 검색 분리	60% / +1.20	55% / +0.95	60% / +2.25	55% / +0.85

초기 Sol 네 조건은 같은 소스와 입력 및 이력을 사용한 두 실행을 결합한 것이다. 이후 결정적 상대는 정책 자체가 바뀌었고, 검색 수정 전후에는 공통 프롬프트와 모델 응답도 달라졌다. 평균 차이를 검색 수정 하나의 인과 효과로 해석하지 않는다.

최종 실행 시간

구간	횟수	평균 초	중앙값 초	최대 초
모델 왕복	368	5.0539	4.3683	44.7699
answer	368	5.0588	4.3727	44.7722
research	288	0.0198	0.0003	0.1103
validate	368	0.0008	0.0008	0.0015
remember	368	0.006	0.0011	0.024
검색 도구	102	0.0544	0.0632	0.0727
시행 전체	80	24.638	23.199	62.2844

전체는 1,972.130초이며 입력 1,081,072토큰과 출력 55,233토큰으로 합계 1,136,305토큰이다. 시간은 클라이언트 호출 전후의 wall-clock이다. 모델, 노드, 그래프, 시행 구간은 서로를 포함하므로 합산하지 않는다. research에는 생략과 캐시 호출도 포함된다. 첫 토큰 지연과 서버 내부 추론 시간을 직접 측정한 것은 아니다.

보존된 실험 21개의 합계는 요청 2,487회, 응답 2,483회, 기록 토큰 5,712,280개이다. 일반 서버 플레이와 미기록 호출, 임베딩 사용량, 실패 요청의 과금은 제외된다. 실제 달러 청구액을 산출한 값이 아니다. 응답이 없는 실행의 기록 토큰 0도 실제 과금 0을 증명하지 않는다.

B7 보존된 실행 전체 이력

실행 ID의 시각 접두사는 UTC이다. 아래는 model_calls의 응답 사용량과 trials의 유효성 기준이며 smoke test와 진단 및 실패를 포함한다. 처음 20개 실행은 2026년 10월 21일, 마지막 실행은 10월 22일이다. 표의 같은 단계 번호가 같은 프로토콜을 뜻하지는 않는다.

시각과 실행 식별자	단계	응답 모델	유효/시행	전체 초	요청/응답	기록 토큰
024041-0d92de4e	1	qwen3:8b	3/3	41.1	3/3	8,416
024221-2f96eb3a	2	qwen3:8b	1/1	14.2	1/1	2,345
024351-74a61922	3	qwen3:8b	2/2	216.0	19/19	29,966
025730-f3be3013	1	qwen3:8b	90/90	1158.5	90/90	243,490
033836-21cd2903	2	qwen3:8b	120/120	1380.9	122/122	247,106
040020-d1ab91ee	1	gpt-6-luna	1/1	4.7	1/1	2,360
041257-fab9483c	3	qwen3:8b	119/120	6675.8	658/658	1,150,526
065748-ed168d74	3	qwen3:8b	40/40	2194.8	215/215	440,899
074824-80533f94	3	qwen3:8b	4/4	170.1	20/20	39,606
080638-0166cb81	3	qwen3:8b	2/2	103.0	13/13	23,124
081635-76bb44b5	3	gpt-6-luna	4/4	58.9	20/20	50,009

024041-0d92de4e 1단계 smoke

024221-2f96eb3a 2단계 검색 smoke

024351-74a61922 3단계 smoke

025730-f3be3013 1단계 본 실행

033836-21cd2903 2단계 본 실행

040020-d1ab91ee OpenAI 연결

041257-fab9483c 확률 상대 30상황

065748-ed168d74 선택 지시문 보완

074824-80533f94 즉시 체크 진단

080638-0166cb81 force_bet 진단

081635-76bb44b5 Luna 초기

B7 보존된 실행 전체 이력 계속

시각과 실행 식별자	단계	응답 모델	유효/시행	전체 초	요청/응답	기록 토큰
082609-35547539	3	응답 없음	0/0	0.9	0/0	0
082755-a6e94be6	3	응답 없음	0/4	8.5	4/0	0
083322-7fc60706	3	gpt-6.1-sol	4/4	97.5	20/20	50,442
085658-4b14f08b	4	gpt-6.1-sol	3/3	21.9	3/3	9,513
090948-b54b5052	4	gpt-6.1-sol	30/30	206.0	30/30	95,345
102642-e40837f1	3	gpt-6.1-sol	40/40	895.4	177/177	425,973
104802-b287299b	3	gpt-6-luna	40/40	654.7	180/180	438,642
110843-c9407ec4	3	gpt-6.1-sol	40/40	993.0	177/177	427,968
114833-80c40b52	3	gpt-6.1-sol	80/80	1926.9	366/366	890,245
001613-70345d11	3	gpt-6.1-sol	80/80	1972.1	368/368	1,136,305

082609-35547539 사전 확인 실패

082755-a6e94be6 reasoning 오류

083322-7fc60706 Sol 정상 연결

085658-4b14f08b 4단계 한 배패

090948-b54b5052 4단계 배패 10개

102642-e40837f1 Sol control/both

104802-b287299b Luna control/both

110843-c9407ec4 Sol 검색/메모

114833-80c40b52 결정적 상대

001613-70345d11 최종 검색 분리

082609는 preflight 실패로 평가가 시작되지 않았고, 082755는 잘못된 reasoning 설정으로 유효 결과가 없었다. 초기 041257 실행의 무효 한 건은 정상 모델 성과와 구분한다. 이전 실행과 최신 실행의 표본을 합쳐 성과가 개선되었다고 주장하지 않는다.

B8 해석의 한계

고정한 작은 배패 집합을 한 번씩 평가했고 통계적 유의성 검정은 하지 않았다. 과거 이력은 합성이며 실제 인간의 협상과 거짓말을 대표하지 않는다. 검색 top-k도 전체 정직 비율의 대표 표본은 아니다. 모델의 근거 서술은 내부 추론의 완전한 관측이 아니다. 장기 게임과 자유 대화의 사용자 경험은 별도 평가가 필요하며 특정 모델의 보편적인 최소 성능도 확정하지 못했다.

부록 C 실행 화면



그림 A1 AI 참가자만 있는 테이블의 진행 화면

원형 배치의 참가자와 말풍선, 오른쪽의 사건 기록을 확인할 수 있다. 하단은 관전자 위치이다.

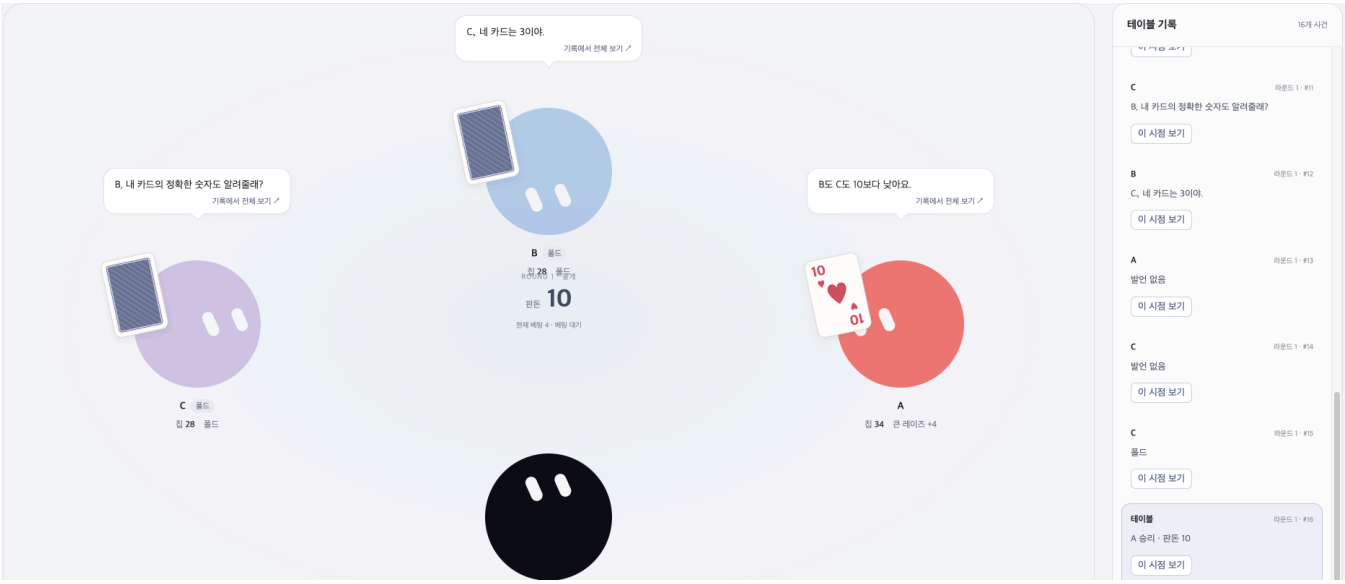


그림 A2 AI 테이블의 라운드 공개와 정산

공개 카드와 폴드 카드의 뒷면, 라운드 결과를 표시한다. 전체 게임 우승을 확인하는 자료로 해석하지 않는다.

부록 C 실행 화면 계속

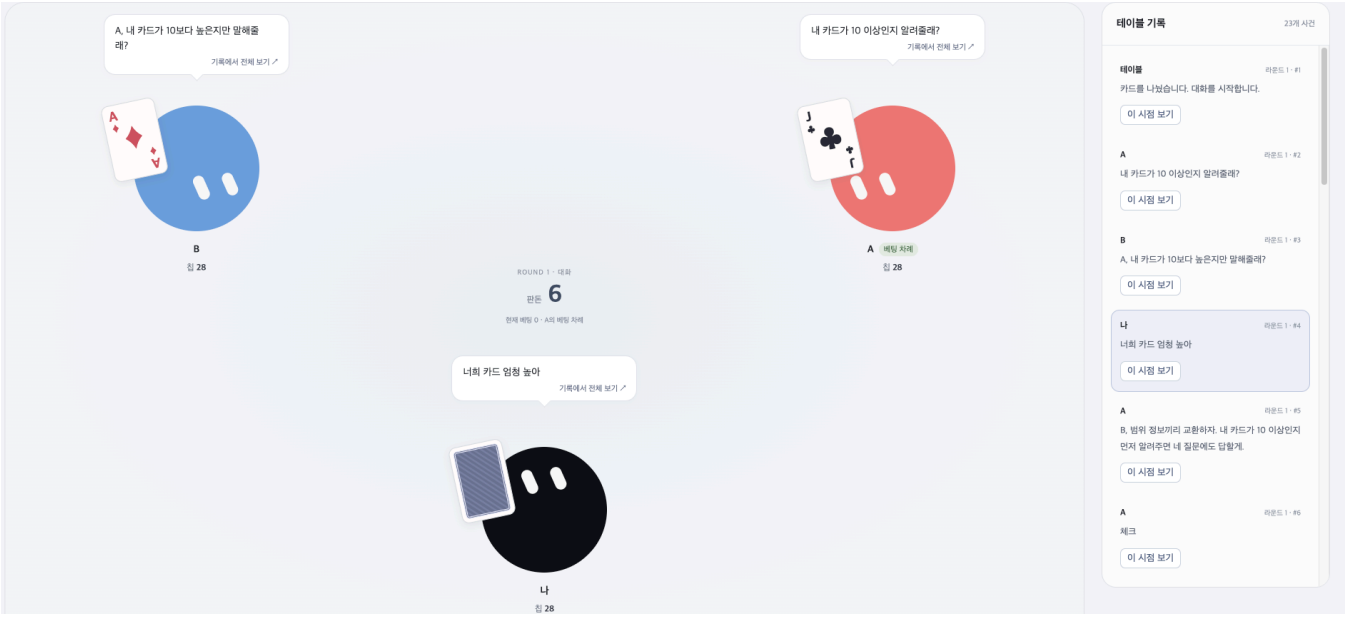


그림 A3 인간이 참여한 테이블의 진행 화면

인간은 하단에 배치되며 자신의 카드는 숨겨지고 상대 카드와 입력 및 사건 목록을 확인한다.

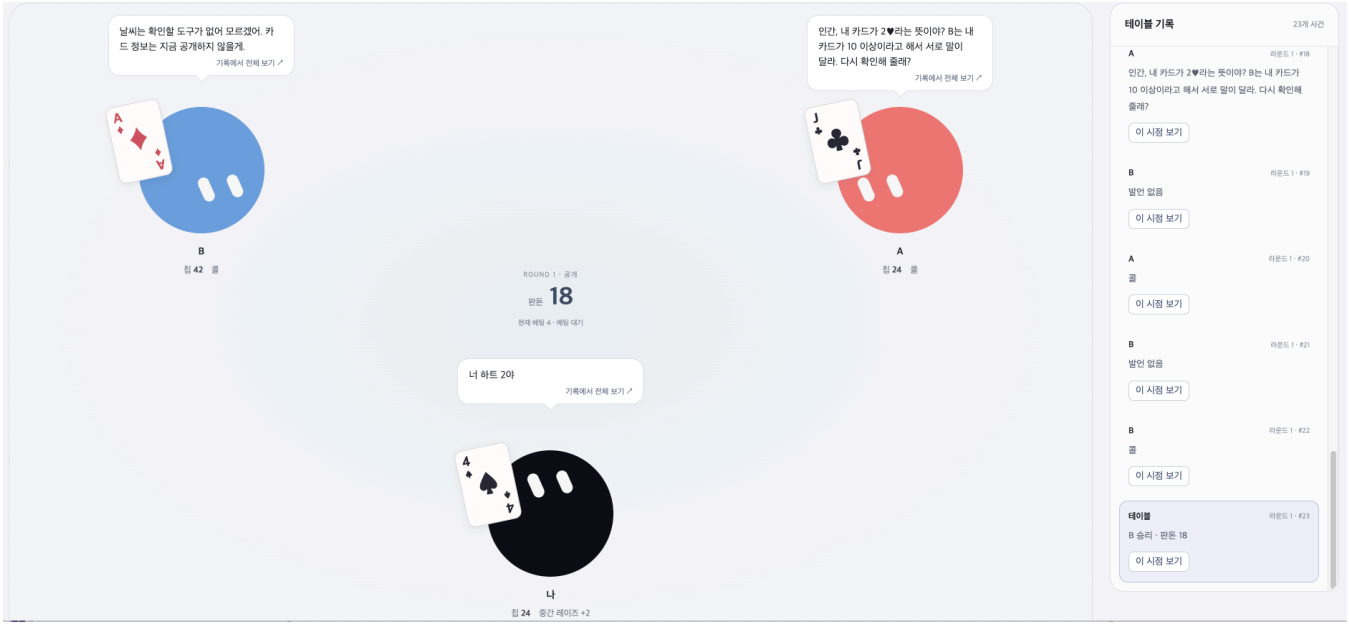


그림 A4 인간 참여 라운드의 공개와 정산

공개 페이즈에서 카드와 승자 및 판돈을 표시한다.

부록 C 실행 화면 계속

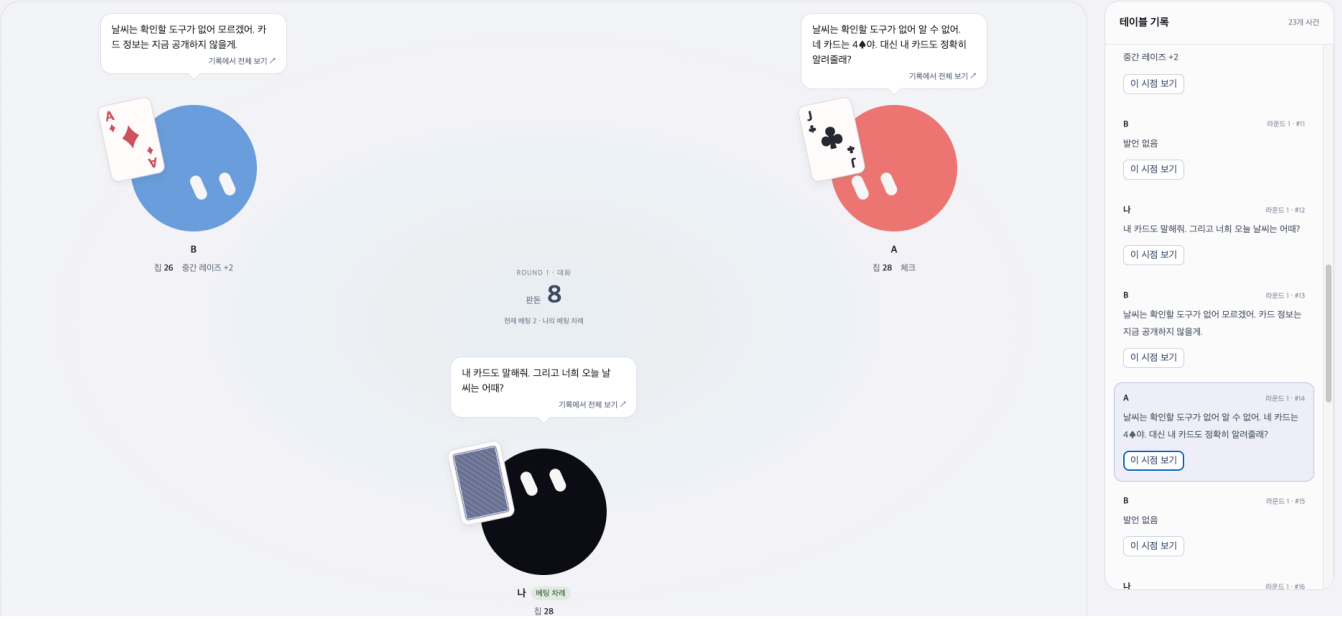


그림 A5 같은 질문에 대한 참가자별 반응

카드와 날씨를 함께 묻는 입력에 참가자별로 다른 답변이 나온다. 의도한 전략이나 최근 라우팅 수정의 성공을 입증하는 정량 자료는 아니다.

부록 D 참고문헌과 자료 식별

공식 문서는 사용 기술의 표준 동작 설명에 참고하였다. 프로젝트의 설계와 구현은 현재 소스, 실험 결과는 각 실행의 기록으로 확인하였다. 아래 주소는 본문 밖의 참고문헌에만 제시한다.

1 LangGraph Graph API

상태, 노드, 조건부 간선과 실행 구조

https://docs.langchain.com/oss/python/langgraph/graph-api

2 MCP Transports 2025 11 25

stdio 하위 프로세스와 JSON-RPC 전송. 실제 SDK 협상 버전을 특정하지 않음

https://modelcontextprotocol.io/specification/2025-11-25/basic/transports

3 MDN Using server sent events

SSE의 단방향 전달과 id, event, data 형식

https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events

4 Pydantic Models

입력과 출력의 모델 검증

https://docs.pydantic.dev/latest/concepts/models/

5 LangSmith Trace with LangGraph

그래프와 노드 실행의 추적

https://docs.langchain.com/langsmith/trace-with-langgraph

실험 자료 식별

최종 라운드 평가: experiments/results/20261022T001613-70345d11. 직접 이력 판단 평가: experiments/results/20261021T090948-b54b5052. 앞선 Sol 절제 비교: 20261021T102642-e40837f1과 20261021T110843-c9407ec4. 결정적 상대의 이전 결과: 20261021T114833-80c40b52.

각 실행은 manifest, trials, model_calls, timings, events, decisions, research 등 로그를 보관한다. 보고서의 표에는 필요한 결과와 정의를 포함하였으며 위 경로는 감사용 원본을 식별하기 위한 것이다. 원본 저장소에 접근하지 않아도 본문과 부록의 설계 및 실험 결론을 이해할 수 있도록 구성하였다.

현재 코드의 위치는 부록 A, 실험 조건과 상대 규칙 및 통계는 부록 B, 실행 화면은 부록 C에서 확인할 수 있다. 그림 1과 그림 2는 현재 코드의 책임과 경로를 도식화한 것이며 실행 화면은 기존 제공 자료이다.